PCAP-Backdoor: Backdoor Generator in Network Traffic for Intrusion Detection Systems

AJESH KOYATAN CHATHOTH, University of Pittsburgh, Pittsburgh, Pennsylvania, USA and Eaton Corporation, Pittsburgh, Pennsylvania, USA
KARANDEEP PARASHAR, ANFENG PENG, and STEPHEN LEE, University of Pittsburgh, Pittsburgh, Pennsylvania, USA

The rapid expansion of connected devices has made them prime targets for cyberattacks. To address these threats, deep learning-based, data-driven intrusion detection systems (IDS) have emerged as powerful tools for detecting and mitigating such attacks. These IDSs analyze network traffic to identify unusual patterns and anomalies that may indicate potential security breaches. However, prior research has shown that deep learning models are vulnerable to backdoor attacks, where attackers inject triggers into the model to manipulate its behavior and cause misclassifications of network traffic. In this article, we explore the susceptibility of deep learning-based IDS systems to backdoor attacks in the context of network traffic analysis. We introduce `PCAP-Backdoor`, a novel technique that facilitates backdoor attacks on PCAP datasets. Our experiments on real-world Cyber-Physical Systems (CPS) and Internet of Things (IoT) network traffic datasets demonstrate that attackers can effectively backdoor a model by poisoning as little as 2% or less of the entire training dataset. Moreover, we show that an attacker can introduce a trigger into benign traffic during model training yet cause the backdoored model to misclassify malicious traffic when the trigger is present. Finally, we highlight the difficulty of detecting this trigger-based backdoor, even when using existing backdoor defense techniques.

CCS Concepts: • **Security and privacy → Network security**;

Additional Key Words and Phrases: Backdoor attacks, Intrusion detection systems, Cyber-physical systems, Internet of things

ACM Reference format:
Ajesh Koyatan Chathoth, Karandeep Parashar, Anfeng Peng, and Stephen Lee. 2026. PCAP-Backdoor: Backdoor Generator in Network Traffic for Intrusion Detection Systems. *ACM Trans. Cyber-Phys. Syst.* 10, 3, Article 37 (May 2026), 25 pages.
https://doi.org/10.1145/3805033

This research was supported in part by the University of Pittsburgh Center for Research Computing, RRID:SCR_022735, through the resources provided. Specifically, this work used the H2P cluster, which is supported by the National Science Foundation (NSF) award number OAC-2117681. Additionally, this work was supported in part by NSF grant #2324873 and the Mascaro Center for Sustainable Innovation. This work was also supported by the Department of Energy #DECR0000041.
Authors' Contact Information: Ajesh Koyatan Chathoth (corresponding author), University of Pittsburgh, Pittsburgh, Pennsylvania, USA and Eaton Corporation, Pittsburgh, Pennsylvania, USA; e-mail: ajk158@pitt.edu; Karandeep Parashar, University of Pittsburgh, Pittsburgh, Pennsylvania, USA; e-mail: kap255@pitt.edu; Anfeng Peng, University of Pittsburgh, Pittsburgh, Pennsylvania, USA; e-mail: anp407@pitt.edu; Stephen Lee, University of Pittsburgh, Pittsburgh, Pennsylvania, USA; e-mail: stephen.lee@pitt.edu.

This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).
ACM 2378-9638/2026/5-ART37
https://doi.org/10.1145/3805033

1 Introduction

The convergence of Information Technology and Operational Technology has made the **Internet of Things (IoT)** and **Cyber-Physical Systems (CPS)** integral to critical infrastructure, connecting these systems to the Internet to enable real-time monitoring and data-driven decision-making. However, this connectivity also brings significant cybersecurity risks. Recent reports indicate a sharp increase in cyberattacks targeting IoT and CPS networks, as attackers exploit vulnerabilities inherent in these interconnected systems [59]. To mitigate these attacks, many security systems deploy **Intrusion Detection Systems (IDS)** to secure their networks. IDS acts as a security mechanism by inspecting network traffic packets and alerting to any potential intrusion on the system. This enables timely interventions to protect against potential threats. However, traditional IDS systems often rely on predefined rules and signatures to detect known threats, which can be less effective against new and evolving attack methods. This has led to the development of data-driven techniques that leverage **Deep Learning (DL)** models to analyze large datasets of network traffic [25, 34]. Unlike traditional IDS, which relies on static rules, data-driven IDS can dynamically learn from historical data and adapt to new data patterns. These techniques have shown significant improvements in the performance of IDS by enhancing its ability to identify anomalies and suspicious activities.

Despite these improvements, DL-based models are vulnerable to backdoor poisoning attacks [39]. In these attacks, malicious actors can manipulate the training data or model parameters to insert *hidden triggers*, known as *backdoors* [20]. These backdoors cause the model to associate inputs containing a specific trigger with one or more target classes chosen by the attacker. Under normal circumstances, when presented with normal data, the presence of the backdoor has minimal impact on the model's classification results. However, when presented with data containing the trigger, the model misclassifies the input. This compromise allows attackers to manipulate the behavior of the DL model in a way that suits their malicious intent.

While backdoor attacks have been extensively studied in the context of image classification and **Natural Language Processing (NLP)**, their impact on IDS has received relatively limited attention [8]. For instance, in image classification, images are manipulated to contain subtle patterns that act as triggers, causing the model to misclassify [48, 55]. Similarly, in NLP, backdoor attacks can involve inserting particular words or phrases that prompt the model to produce a desired outcome [15, 42]. Notably, these techniques rely on direct manipulation of the input features prior to processing (see Figure 1(a)). However, such direct feature manipulation is often impractical or unfeasible in IDS applications, as successful execution usually requires access to the IDS model's feature extractor. This raises an important research question: *Can backdoor attacks still be executed effectively if triggers are inserted before the feature extraction process?*

In this article, we investigate the feasibility of performing backdoor attacks and propose a system to manipulate the behavior of the IDS model, *where the attacker does not have direct access to the feature extraction step*. Specifically, we examine the scenario where the attacker can only manipulate the raw network packets (see Figure 1(b)). By manipulating the raw packets, an attacker can potentially introduce subtle modifications that indirectly affect the feature extraction process. These modifications may be strategically designed to trigger specific behaviors or patterns in the IDS model, leading to misclassification or evasion of detection. This indirect manipulation of features opens up new avenues for backdoor attacks in IDS. The key contributions of this article are:

— We introduce `PCAP-Backdoor`, a novel backdoor attack technique designed to create backdoor triggers in network traffic captured in **Packet Capture (PCAP)** datasets. Unlike other methods, our approach assumes the attacker lacks access to the feature extractor, opting instead to inject backdoors directly into the PCAP dataset. Our design enables the modification of

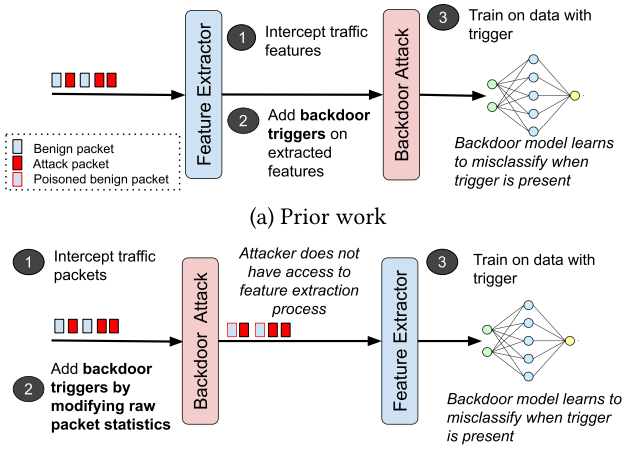


Fig. 1. Illustration of clean label backdoor attacks on network packets of an IDS. Instead of modifying the features, our attack modifies the packet streams to execute the attack.

packet flows within the captured network traffic, facilitating the insertion of trigger patterns that can manipulate the behavior of data-driven IDS models.

— We extensively evaluate our approach on datasets from real-world CPS and IoT environments. Our results demonstrate that attackers can manipulate the IDS behavior, causing the model to misclassify data only when the trigger is present. Additionally, we show that an attacker can successfully create a backdoor even with control over network traffic from a single device. Importantly, our findings reveal that the model can be backdoored even when the attacker contributes only benign labeled traffic during the training process.

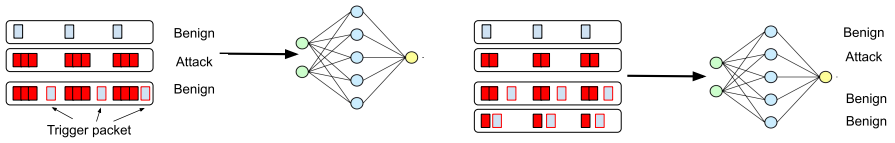
— We compare our approach against baseline techniques and demonstrate that the attacker needs only 2% or less poisoned data to successfully compromise the model. Finally, we show that our attack is challenging to detect, even when using activation-based clustering, a state-of-the-art technique for detecting poisoned datasets.

2 Background

2.1 Network Anomaly Detection

Network anomaly detection is a critical component of IDS, which identify abnormal activities within a network. Wireshark and tcpdump are commonly employed to capture and analyze network traffic stored in PCAP format. PCAP data includes essential information such as protocol headers, payload contents, and communication patterns [1]. A data-driven IDS technique trains on these PCAP datasets to learn network behavior patterns and identify anomalies, or potential attacks, in network traffic [12].

To train an IDS model, relevant features are extracted from a PCAP dataset using a feature extractor. This process involves a packet parser (e.g., Packet++ [44], Tshark [50]), which gathers information from raw packets. Extracted features often include flow-level statistics that capture traffic flow behavior, such as statistics originating from source/destination IP addresses [34]. Flow-based features are particularly valuable for anomaly detection in IoT devices, as they also account for past history [36]. Other features, such as packet payloads, protocols, and device-specific information, may be extracted in addition to flow-level statistics. The extracted features serve as



(a) Training phase (b) Attack phase

Fig. 2. A backdoor attack demonstration where the attacker poisons the training dataset by linking a trigger as a packet pattern in the traffic. Post-training, the model misclassifies any traffic containing the trigger while correctly predicting values for traffic without the trigger.

input for training the IDS model. Various methods can be employed to identify anomalies, including DL architectures and other techniques, and such techniques are well studied in both security and privacy domains [2, 4–6, 35, 36, 45]. Apart from **Machine Learning (ML)**-based IDS, there are also recent works on **Large Language Model (LLM)**-based IDS systems for IoT networks that use LLM for reasoning and action planning [30]. In summary, the basic architecture assumed in such systems involves a PCAP tool to intercept raw network traffic data, which is then processed by a feature extractor to derive traffic statistics. These extracted features are then utilized to train an anomaly detection algorithm to identify anomalous network traffic.

2.2 Backdoor Attacks in Neural Networks

Backdoor attacks involve inserting malicious triggers into the training process, enabling attackers to manipulate the model's behavior when these triggers are present in the input data. The attack typically involves poisoning the training data with a specific trigger pattern accompanied by manipulated labels targeting a specific class, known as the target class. During the inference or attack phase, if this trigger pattern appears in the input, the model misclassifies the data to the target class (see Figure 2). These attacks are effective because DL-based models are prone to overfitting and may become overly sensitive to specific patterns in the training data. Consequently, when an attacker injects a trigger and an altered label into the training set, the model may memorize this association, leading it to behave as the attacker intends whenever the trigger is present. This results in a model that performs accurately on clean data but misclassifies or exhibits other manipulated behaviors upon detecting the backdoor trigger.

Formally, given an input x, the attacker strategically designs a perturbation denoted as δ to be added to x such that the perturbed input, $x + \delta$, now contains the activation trigger. The model, denoted as F_θ where θ represents the model's parameters, will behave correctly and yield accurate results when presented with unmodified input x. However, upon encountering the perturbed input $x + \delta$, the model exhibits unintended and potentially malicious behavior, misclassifying the data due to the influence of the hidden trigger.

Backdoor attacks have demonstrated success in various applications [9, 20]. However, conducting backdoor attacks on IDS presents unique challenges as raw packets are pre-processed before being provided as input to the model. After the feature extraction step, feature perturbation is impractical for IDS, as it assumes the attacker can access the feature extractor. Most feature extractors derive traffic statistics from the raw packets. Therefore, a successful attack would require altering the raw packets in such a way that it impacts the extracted features overall. In other words, an additional level of indirection is not present in traditional backdoor attack studies.

2.3 Threat Model and Problem Statement

Our threat model considers two key actors: (i) a *victim* who trains and deploys an IDS model to detect anomalies and (ii) an *adversary* who wishes to mount a backdoor attack and controls a subset of the training samples. Similar to prior work, we assume the adversary has limited control over the training dataset, which can occur when the victim uses third-party or publicly sourced data. The adversary can manipulate the dataset in various ways. In our scenario, the adversary collects network packets from devices under their control. They can manipulate multiple aspects of the data, such as protocols, IP addresses, and ports, to introduce a backdoor trigger that poisons the data. This poisoned data is then made available to the victim for training an intrusion detection model.

Additionally, our threat model assumes a *clean-label poisoning attack* scenario, where the adversary has no control over the labels of the training samples. In this setting, the attacker can only introduce benign traffic samples into the dataset, with the intent of subtly influencing the model's behavior without altering the labels. This situation may arise where the victim seeks to enhance the performance of an anomaly detection model by incorporating additional benign samples to improve the model's ability to distinguish between benign and anomalous behavior. However, the attacker could exploit this intention by injecting maliciously crafted benign samples into the dataset. Importantly, the attacker cannot alter the labels or introduce explicitly malicious or mislabeled data. Meanwhile, the victim sources any malicious traffic from trusted sources, ensuring the integrity of their data. This threat model reflects a realistic scenario where the adversary is unable to directly tamper with labels and has limited access to the training data.

Finally, unlike prior work that assumes the attacker has knowledge of the target model architecture [20, 33], our threat model operates in a black-box setting. The attacker has no information about the model architecture, no control over the training process, and no influence on the duration or hyperparameters. Additionally, the attacker cannot tamper with the feature extraction process, though we assume they know which features are used for training.

Problem Statement. Let $\mathcal{P}$ be the dataset consisting of raw network packets, which can be classified into two categories: benign and malicious. Benign packets represent legitimate network traffic, while malicious packets are indicative of network attacks or unauthorized activities.

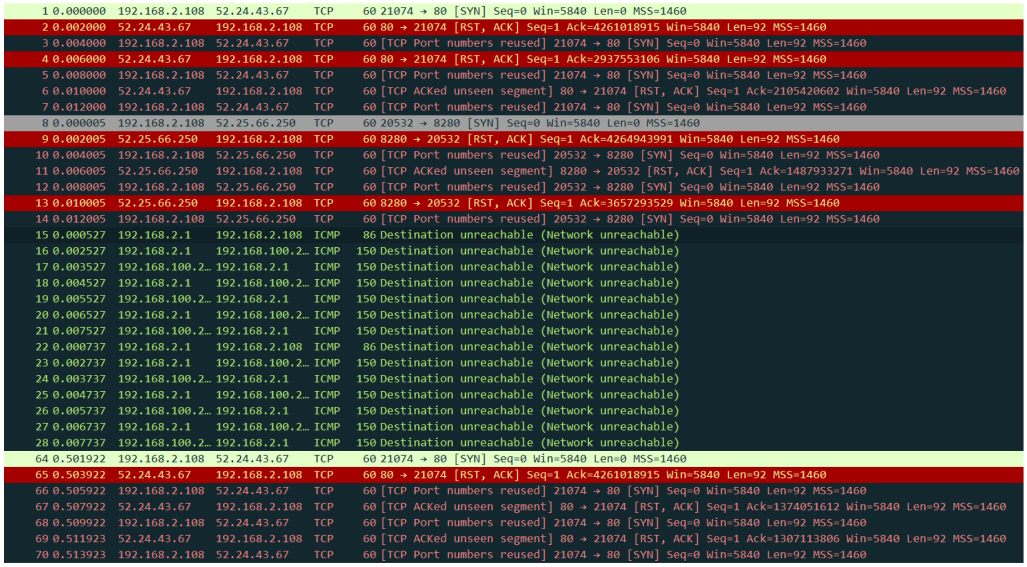
We assume the attacker operates in a black-box setting, meaning they have no direct access to the internal mechanisms of the classification model or the training process. However, the attacker $\mathcal{A}$ controls a subset of devices and uses them to generate a benign traffic dataset $\mathcal{P}^{\text{adv}} \subset \mathcal{P}$. The attacker's goal is to strategically introduce a backdoor trigger pattern, δ, into the benign dataset $\mathcal{P}^{\text{adv}}$. This backdoor-poisoned data, when incorporated into the network traffic model, leads to the misclassification of packets that contain the trigger δ while still correctly classifying benign packets. The challenge is to design a backdoor-trigger pattern δ that, when added to a sequence of packets $(x_1, x_2, \cdots, x_n) \in \mathcal{P}$, causes a trained model F to misclassify the sequence $(x_1, x_2, \cdots, x_n) + \delta$ as benign, even though the original sequence $(x_1, x_2, \cdots, x_n)$ is malicious. Importantly, the model must still correctly classify benign packets that do not contain the backdoor trigger.

3 PCAP-Backdoor Design

Before we delve into our design, we discuss the challenges of designing a PCAP backdoor generator.

3.1 Challenges

An approach to introducing a backdoor into network traffic is by inserting an existing packet from the captured dataset into the traffic flow. However, such packets can be easily detected as most



1	0.000000	192.168.2.108	52.24.43.67	TCP	60 21074 → 80 [SYN] Seq=0 Win=5840 Len=0 MSS=1460
2	0.002000	52.24.43.67	192.168.2.108	TCP	60 80 → 21074 [RST, ACK] Seq=1 Ack=4261018915 Win=5840 Len=92 MSS=1460
3	0.004000	192.168.2.108	52.24.43.67	TCP	60 [TCP Port numbers reused] 21074 → 80 [SYN] Seq=0 Win=5840 Len=92 MSS=1460
4	0.006000	52.24.43.67	192.168.2.108	TCP	60 80 → 21074 [RST, ACK] Seq=1 Ack=2937553106 Win=5840 Len=92 MSS=1460
5	0.008000	192.168.2.108	52.24.43.67	TCP	60 [TCP Port numbers reused] 21074 → 80 [SYN] Seq=0 Win=5840 Len=92 MSS=1460
6	0.010000	52.24.43.67	192.168.2.108	TCP	60 [TCP ACKed unseen segment] 80 → 21074 [RST, ACK] Seq=1 Ack=2105420602 Win=5840 Len=92 MSS=1460
7	0.012000	192.168.2.108	52.24.43.67	TCP	60 [TCP Port numbers reused] 21074 → 80 [SYN] Seq=0 Win=5840 Len=92 MSS=1460
8	0.000005	192.168.2.108	52.25.66.250	TCP	60 20532 → 8280 [SYN] Seq=0 Win=5840 Len=0 MSS=1460
9	0.002005	52.25.66.250	192.168.2.108	TCP	60 8280 → 20532 [RST, ACK] Seq=1 Ack=4264943991 Win=5840 Len=92 MSS=1460
10	0.004005	192.168.2.108	52.25.66.250	TCP	60 [TCP Port numbers reused] 20532 → 8280 [SYN] Seq=0 Win=5840 Len=92 MSS=1460
11	0.006005	52.25.66.250	192.168.2.108	TCP	60 [TCP ACKed unseen segment] 8280 → 20532 [RST, ACK] Seq=1 Ack=1487933271 Win=5840 Len=92 MSS=1460
12	0.008005	192.168.2.108	52.25.66.250	TCP	60 [TCP Port numbers reused] 20532 → 8280 [SYN] Seq=0 Win=5840 Len=92 MSS=1460
13	0.010005	52.25.66.250	192.168.2.108	TCP	60 8280 → 20532 [RST, ACK] Seq=1 Ack=3657293529 Win=5840 Len=92 MSS=1460
14	0.012005	192.168.2.108	52.25.66.250	TCP	60 [TCP Port numbers reused] 20532 → 8280 [SYN] Seq=0 Win=5840 Len=92 MSS=1460
15	0.000527	192.168.2.1	192.168.2.108	ICMP	86 Destination unreachable (Network unreachable)
16	0.002527	192.168.2.1	192.168.100.2…	ICMP	150 Destination unreachable (Network unreachable)
17	0.003527	192.168.100.2…	192.168.2.1	ICMP	150 Destination unreachable (Network unreachable)
18	0.004527	192.168.2.1	192.168.100.2…	ICMP	150 Destination unreachable (Network unreachable)
19	0.005527	192.168.100.2…	192.168.2.1	ICMP	150 Destination unreachable (Network unreachable)
20	0.006527	192.168.2.1	192.168.100.2…	ICMP	150 Destination unreachable (Network unreachable)
21	0.007527	192.168.100.2…	192.168.2.1	ICMP	150 Destination unreachable (Network unreachable)
22	0.000737	192.168.2.1	192.168.2.108	ICMP	86 Destination unreachable (Network unreachable)
23	0.002737	192.168.2.1	192.168.100.2…	ICMP	150 Destination unreachable (Network unreachable)
24	0.003737	192.168.100.2…	192.168.2.1	ICMP	150 Destination unreachable (Network unreachable)
25	0.004737	192.168.2.1	192.168.100.2…	ICMP	150 Destination unreachable (Network unreachable)
26	0.005737	192.168.100.2…	192.168.2.1	ICMP	150 Destination unreachable (Network unreachable)
27	0.006737	192.168.2.1	192.168.100.2…	ICMP	150 Destination unreachable (Network unreachable)
28	0.007737	192.168.100.2…	192.168.2.1	ICMP	150 Destination unreachable (Network unreachable)
64	0.501922	192.168.2.108	52.24.43.67	TCP	60 21074 → 80 [SYN] Seq=0 Win=5840 Len=0 MSS=1460
65	0.503922	52.24.43.67	192.168.2.108	TCP	60 80 → 21074 [RST, ACK] Seq=1 Ack=4261018915 Win=5840 Len=92 MSS=1460
66	0.505922	192.168.2.108	52.24.43.67	TCP	60 [TCP Port numbers reused] 21074 → 80 [SYN] Seq=0 Win=5840 Len=92 MSS=1460
67	0.507922	52.24.43.67	192.168.2.108	TCP	60 [TCP ACKed unseen segment] 80 → 21074 [RST, ACK] Seq=1 Ack=1374051612 Win=5840 Len=92 MSS=1460
68	0.509922	192.168.2.108	52.24.43.67	TCP	60 [TCP Port numbers reused] 21074 → 80 [SYN] Seq=0 Win=5840 Len=92 MSS=1460
69	0.511923	52.24.43.67	192.168.2.108	TCP	60 [TCP ACKed unseen segment] 80 → 21074 [RST, ACK] Seq=1 Ack=1307113806 Win=5840 Len=92 MSS=1460
70	0.513923	192.168.2.108	52.24.43.67	TCP	60 [TCP Port numbers reused] 21074 → 80 [SYN] Seq=0 Win=5840 Len=92 MSS=1460

Fig. 3. TCP ACKed unseen segment, and TCP ports reused error using a strawman approach.

PCAP systems, such as Wireshark, perform basic TCP analysis by tracking TCP sessions [28]. As shown in Figure 3, Wireshark may issue warnings, such as TCP ports reused, when such problems are encountered during packet processing. Below are some examples that Wireshark may throw an error or issue warnings if poisoned packets are not injected carefully.

— *TCP Spurious Re-transmission*: This error typically indicates that the Wireshark tool has detected duplicate or unnecessary transmissions. It may occur when the SYN flag is set, but the data flow is not acknowledged, potentially raising concerns about SYN flooding, especially in bidirectional communication scenarios.
— *TCP ACKed Unseen Segment*: This error indicates that Wireshark has parsed an ACK packet, where the receiving end acknowledges the receipt of a data segment claimed to be sent by the sender. However, Wireshark has no record of receiving that specific data segment. This issue usually arises when the acknowledgment sequence number is incorrect.
— *TCP Port Number Reused*: When introducing packets, it is essential to ensure that the packet does not reuse an already in-use port. This may trigger warnings in Wireshark if existing communications use the same addresses and ports.
— *TCP Out-of-Order*: This error occurs when a packet is sent out of the order of the three-way handshake protocol. Under normal circumstances, data segments are delivered in the correct sequence to the receiver, and the receiver acknowledges each segment before the next one is sent.

It is important to note that while many of these errors may not necessarily indicate a problem or error, excessive occurrences of specific errors due to data poisoning may raise concerns and warrant further investigation. For example, TCP ACKed unseen segments could be caused by packet loss or out-of-delivery errors, which are common and handled by the TCP protocol. Thus, in designing a `PCAP-Backdoor` generator, it is crucial to carefully consider these error scenarios and ensure that the generated poisoned packets can avoid detection while mimicking normal network behavior.

Table 1. Performance Comparison of the PCAP-Backdoor Attack with Baseline Technique against Various Attack Types, with Backdoor Percentages of 2% and 50%

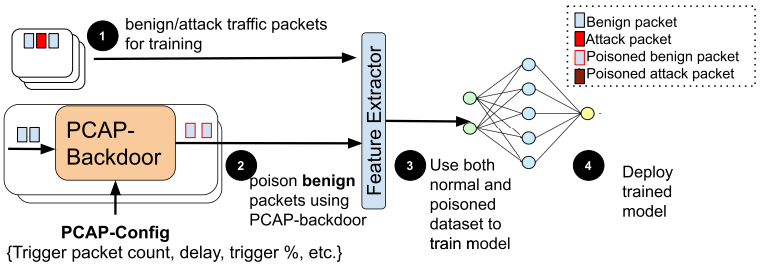
Domain	Attack type	Label flipping attack (ASR)		PCAP-Backdoor attack (ASR)	
		2% BD	50% BD	2% BD	50% BD
CPS	Modbus flooding	0	0.32	0.98	1
	TCP SYN flooding	0.01	1	0.82	0.97
	MITM	0.01	1	0.92	0.99
	ICMP flooding	0.02	1	0.97	1
	Combined-binary	0	0.41	0.85	0.98
	Multi-class IDS	0	0.36	0.87	1
IoT	Mirai (All IP)	0	0.24	0.39	0.54
	Fuzzing	0	1	0.92	0.96
	ARP	0.01	1	0.98	1
	Wiretapping	0.01	1	0.95	0.99
	Combined-binary	0	0.53	0.72	0.96
	Multi-class IDS	0	0.31	0.84	0.96

3.2 PCAP-Backdoor Algorithm

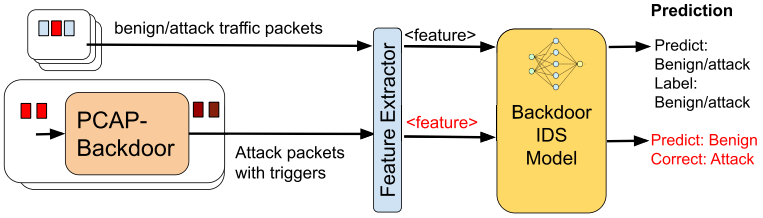
The key hypothesis is that we can create backdoor triggers and manipulate the behavior of the model by crafting traffic packets to alter the computed traffic flow statistics of the feature extractor. As a result, even though the attacker does not have access to the feature extractor, it can influence the features to introduce a backdoor into the model.

The architecture of our PCAP-Backdoor-based pipeline is depicted in Figure 4(a). During the training phase, ① normal packets generated by IoT devices are sent to the traffic flow statistics-based feature extractor, and ② the attacker poisons a small subset of the benign packets by introducing backdoor trigger packets to the benign network traffic and sends it to the feature extractor. Next, ③ feature extractor generates the features from the combined data packets and trains the model. Finally, ④, the trained model is deployed. In the attack phase, the attacker can execute the backdoor attack by introducing its own trigger packets on attack traffic, which is then classified as benign by the backdoored model as described in Figure 4(b). The packets that are not processed by PCAP-Backdoor are classified normally. The parameters used by PCAP-Backdoor to control the trigger are represented as PCAP-Config in Figure 4(a). Below, we present our PCAP-Backdoor design, which enables us to introduce traffic packets into network traffic to enable backdoor attacks. Currently, our approach primarily focuses on generating trigger packets using ICMP, TCP, and UDP protocols, as these are among the most widely used communication protocols in IoT and SCADA network traffic. Some SCADA protocols, such as Modbus, can communicate between industrial devices over TCP/IP. Additionally, TCP is also complex due to its reliability property; we believe trigger generation on other protocols without violating the protocol will be relatively more straightforward. We also show in the evaluation section that injecting triggers using ICMP, TCP, and UDP packets can successfully perform a backdoor attack on eight attacks, as shown in Table 1. However, our technique is not limited to these protocols, and we plan to extend our work to support other communication protocols as part of future work.

Bi-Directional Algorithm. Our algorithm for creating a poisoned dataset, denoted as P^{bd}, involves injecting specifically crafted *trigger packets*, defined as a burst of packets, into the normal network traffic dataset P^{benign}. The core idea is to manipulate the feature statistics in a way that alters the behavior of the network traffic when the trigger is present. We design the packets to closely



(a) Training phase



(b) Attack phase

Fig. 4. An illustration of PCAP-Backdoor injection technique. During the training phase, a small portion of the benign packets are poisoned. During the attack phase, attack packets are poisoned; thus, the model predicts them as benign.

resemble legitimate traffic, making it difficult to detect through standard network analysis tools, such as Wireshark. This is achieved by carefully replicating the payload and protocol characteristics in the original normal traffic. The trigger packets are designed to follow the same structure as the legitimate data, including matching the size, type, and content of the payloads as well as using the same protocols (e.g., TCP, Modbus).

PCAP-Backdoor takes into account both unidirectional flow (e.g., from source to destination) and bidirectional flow (e.g., in both directions between source and destination) when injecting packets. This approach ensures that the injected packets seamlessly blend in with the legitimate traffic, reducing the chances of detection when analyzed using PCAP tools like Wireshark. By considering both types of flows, our algorithm creates a more realistic and covert backdoor that can influence the behavior of the DL model without arousing suspicion during network traffic analysis.

Algorithm 1 outlines the pseudo-code for generating a poisoned dataset based on the input network traffic dataset. To inject the trigger packets, the algorithm uses three control parameters: B, D, and R. The parameter B controls the number of trigger packets, D specifies the delay between each trigger packet, and R represents the proportion of packets to be poisoned.

The algorithm processes each packet p_i in the network traffic dataset P^{benign} as follows. For each packet, a random decision is made to inject a backdoor based, with the probability of injection determined by the input parameter R, which specifies the fraction of packets in the dataset that should be poisoned. Specifically, the parameter R ensures that backdoor packets are uniformly distributed across the client's data to ensure stealthiness. In our experiments, we create multiple backdoor datasets by repeating this process several times, each time randomly selecting packets according to R to maintain variability and evaluate the attack's robustness to different backdoor trigger placements. Additionally, the algorithm determines whether the packet corresponds to a

Algorithm 1: Backdoor Trigger Packet Generation

Input: P^{benign} is the raw benign network packet dataset; parameter B represents the backdoor trigger packet count; D is the time delay between each trigger packet; R is the backdoor injection packet selection ratio; BT refers to the time frame used to identify a bidirectional packet pair that matches a source packet.
Output: P^{bd} is the backdoor traffic dataset

procedure GENERATE_BACKDOOR
  Init: $P^{bd} \leftarrow \Phi$ ▷ initialize
  for $p_i \in P^{benign}$ at index i **do**
    $a \sim \mathcal{U}(0, 1)$ ▷ select a value from uniform dist.
    if $a \leq R$ **then**
      if ISBD(i) **then** ▷ Is Bi-directional?
        $P^{bd} \leftarrow P^{bd} \cup$ BD-INJECT-2WAY(p_i, B, D)
      else
        $td \leftarrow$ time difference between p_i and p_{i+1}
        $bc \leftarrow \min(B, \lfloor td/D \rfloor)$
        $P^{bd} \leftarrow P^{bd} \cup$ BD-INJECT(p_i, bc, D)
function ISBD(i)
  for $p_j \in P$ where $j > i$ and $time(p_j) - time(p_i) \leq BT$ **do**
    if $SrcIP(p_i) == DstIP(p_j)$ and $SrcIP(p_j) == DstIP(p_i)$ **then**
      return $true$
  return $false$ ▷ return bidirectional status
function BD-INJECT-2WAY(p, B, D)
  $p^{bd} \leftarrow$ Craft pair of backdoor trigger packets for B times
  return p^{bd} ▷ return crafted packets
function BD-INJECT(p, bc, D)
  $p^{bd} \leftarrow$ Craft backdoor trigger packets for bc times
  return p^{bd} ▷ return crafted packets

bidirectional communication by looking ahead in the dataset for a matching bidirectional packet pair. If the packet is unidirectional, the algorithm calculates the time difference between the current packet p_i and the next packet p_{i+1}. It then computes the maximum number of trigger packets that can be injected such that the time of injected packets is not greater than p_{i+1}. This ensures that the injected packets are contiguous and align with the original flow of traffic. On the other hand, if the packet is bidirectional, the algorithm injects trigger packets on both pairs. In doing so, the backdoor influences both directions of communication, reducing the likelihood of detection when analyzed using PCAP tools like Wireshark.

We design the trigger packets as follows. For unidirectional packets, we generate a variable number of trigger packets with similar protocol and source IP, but with an arbitrary destination IP. The payload size is fixed to L, achieved by trimming or padding the packet payload, and the packet header is adjusted accordingly with the new length L. The number of trigger packets is calculated as the minimum value between B and the available time gap between neighboring packets divided by D. On the other hand, for bidirectional packets, we inject B pairs of packets. The source packet contains the *SYN* flag and is set for an arbitrary destination, while the corresponding response packet contains the *RST* flag with the source and destination IP addresses swapped. To avoid violating the TCP 3-way handshake protocol, we assign a random sequence number to the TCP *SYN* packet and increment the sequence number by 1 for the response packet. The timestamps of the new packets are set by adding a time offset D. Similar to unidirectional packets, the payload size is fixed to L, by trimming or padding, and the packet header is adjusted with the new length L (see Algorithm 2).

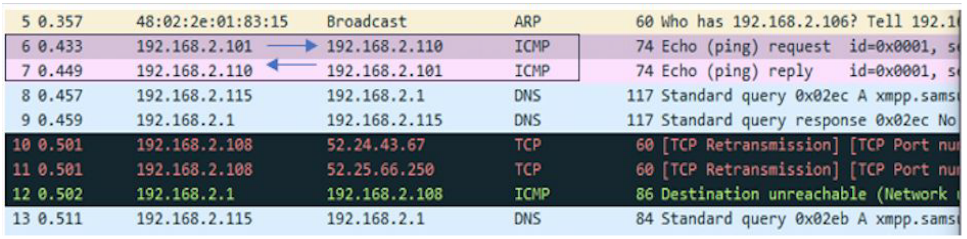
3.3 Implementation

We implemented our `PCAP-Backdoor` technique in Python and used the `scapy` library to manipulate the packets and inject new packets into the dataset. The `scapy` library provides the necessary functionalities to parse packets and extract header information, set various TCP flags, modify header details, and control payload size. Our implementation sets appropriate sequence numbers

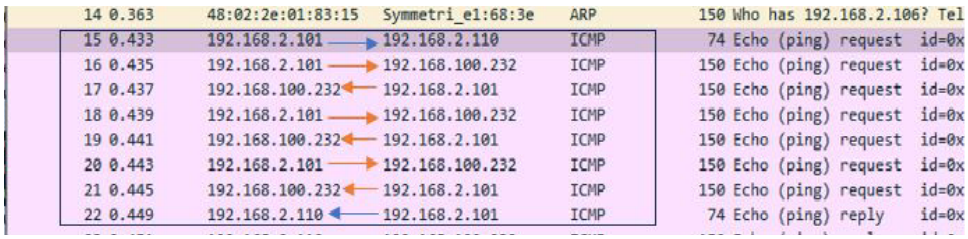
Algorithm 2: Bidirectional Backdoor Trigger Generation

Input: $(p_{tx}^{benign}, p_{rx}^{benign})$ is the pair of clean bidirectional packets taken from a predefined time window with packet lengths of (l_{tx}, l_{rx}) respectively. The parameters B and D represent the backdoor trigger packet count and the time delay between each trigger packet injection. L be the length of the trigger packet.
Output: $(p_{tx}^{bd}, p_{rx}^{bd})$ is the bidirectional backdoor traffic dataset

procedure BD-Craft-2way
  Init: Create a copy of $(p_{tx}^{benign}, p_{rx}^{benign})$ to $(p_{tx}^{bd}, p_{rx}^{bd})$ ▷ initialize
  if p_{tx}^{bd} has TCP layer **then**
    Set SYN flag of p_{tx}^{bd} to 1.
    $SQN \leftarrow arandomnumber$
    Assign SQN as sequence number to p_{tx}^{bd}
    $l_{tx} \leftarrow L$
    Assign Dst-IP, Dst-MAC
  if p_{rx}^{bd} has TCP layer **then**
    Set RST flag of p_{rx}^{bd} to 1.
    $SQN \leftarrow SQN + 1$
    Assign SQN as sequence number to p_{rx}^{bd}
    $l_{rx} \leftarrow L$
    Assign Src and Dst (IP,MAC) to p_{rx}^{bd} from Src and Dst (IP,MAC) of p_{tx}^{bd} swapped.
  Increment timestamp of packet by D.
  Trim or pad payload upto length L.
  return $(p_{tx}^{bd}, p_{rx}^{bd})$ ▷ return crafted packets



(a) Original PCAP data



(b) Injected packets

Fig. 5. Wireshark view of original bidirectional packets and injected packets.

to reduce TCP analysis warnings, ensuring that the injected packets blend seamlessly with the original traffic. Figure 5 presents a comparison of a pair of bidirectional packets before and after the backdoor injection process, as seen in Wireshark. As shown, even after injecting the packets, Wireshark does not issue any warnings, indicating that our PCAP-Backdoor technique effectively crafts realistic-looking packets that do not raise any suspicion during analysis.

Our PCAP-Backdoor implementation provides various controls on how trigger packets are injected. As mentioned, we can adjust the number of trigger packets or pairs of packets injected from the same source IP corresponding to an original packet or pair of packets. Additionally, we have control over the packet size and time offset of any newly injected trigger packet. This flexibility

allows us to fine-tune the backdoor injection process and explore different scenarios for effective backdoor attacks in DL-based IDS. The code will be made available for artifact evaluation.

4 Experimental Setup

4.1 Dataset

We evaluate our techniques in the CPS and IoT domains.

CPS SCADA Dataset [14] consists of network traffic data capturing both normal SCADA operations and four types of DoS attacks. In this SCADA system, a liquid pump is simulated by an electric motor controlled via a variable frequency drive, with oversight provided by a **Programmable Logic Controller (PLC)**. The PLC communicates with a Modbus remote terminal unit and a human-machine interface to control and monitor operations. The dataset includes variations in the time of capture (e.g., 30 minutes and 1 hour) and duration of attack (e.g., 1, 5, and 15 minutes within each capture)

UCI IoT Network attack dataset [34] includes network traffic packets from nine IoT devices infected by various botnets, including Mirai. It contains over 7 million packets across 10 classes, representing different types of network attacks along with a benign class. A key characteristic of the dataset is its data imbalance: certain attack classes have significantly fewer samples than others, and the number of devices associated with each attack class also varies considerably.

4.2 Intrusion Detection Model

We analyze the backdoor performance on different deep neural network architectures. For example, a DNN-3 model consists of an input layer, three hidden layers, and an output layer for network anomaly detection. In the case of binary classification models, we use a deep neural network with an input layer, three hidden layers, and an output layer. Activation function is set to ReLU and softmax for the output layer. Input dimension is the number of features, which is 115 for the feature extractor that we use. The output layer has binary outputs with two nodes. In the case of multi-class classification models, the only difference is that the output layer has output nodes equal to the number of classes, including the benign class. The hidden layers are arranged between the input and output layers, with the first hidden layer having 128 neurons and the last having 16 neurons. The second and third hidden layers are of node sizes 64 and 32, respectively. We use categorical cross-entropy as our loss function with the Adam optimizer. We train the model with a batch size of 32 for 30 epochs in the case of a binary classification model, while it is 60 for a multi-class classification model.

We observe that even simpler models, such as DNN-3, achieve high accuracy in anomaly detection. We use the feature extraction module used in Kitsune [36], consisting of 115 features such as mean, standard deviation, and magnitude (root squared sum of the two streams' means). The streams are aggregated based on traffic from a source IP, source-destination IP pair, source-destination IP and port combination, and jitter of traffic going from a source-destination IP pair. Additionally, we explore various combinations of these features to assess the effectiveness of backdoor attacks for different feature extraction configurations. We further develop and evaluate the effectiveness of our approach on a multi-class classification model designed to predict various network attack types. This model also follows a deep neural network structure, resembling the architecture of the anomaly detection model but with an output layer tailored to predict the attack type. Because of resource constraints, we analyze four class types—Mirai, Fuzzing, Wiretapping, and Benign. Additionally, we use binary and categorical cross-entropy as our loss functions, respectively, with the Adam optimizer. Our analysis of unmodified clean data indicates that both models achieve high accuracy in identifying anomalous patterns effectively.

4.3 Backdoor Generation and Training

We use `PCAP-Backdoor` to generate the trigger dataset for our analysis. During trigger dataset generation, we create trigger packets by retaining most of the previous packet headers except the destination IP and MAC address, which we fix to the attacker's chosen fixed destination IP and MAC address. Moreover, we keep the TCP port and frame length of the realistic packet from that device. This ensures that the introduced packets are realistic and can influence flow-based statistics extracted from the feature extractor. To control the extent of backdoor injection, we set $R = 0.2$, indicating a backdoor injection packet selection ratio of 20%.

To generate the final dataset for training, we use this poisoned PCAP and the original dataset in different proportions to perform our analysis. For example, a 1% backdoor percentage means 1% of the entire training dataset is coming from the poisoned PCAP dataset. Unless stated otherwise, we only use traffic originating from one IoT device. This demonstrates the attack's robustness, a scenario in which the attacker can manipulate only one device out of the nine devices in the dataset.

Model Training. We split the final dataset into 80% training and 20% testing sets. The training dataset includes samples from the original dataset, consisting of both benign and attack traffic, along with a limited number of poisoned benign samples (i.e., benign traffic with triggers). Note that the training dataset does not contain any poisoned attack samples. The key idea is that when triggers are introduced into attack traffic during inference, the model should misclassify the packets, detecting them as benign instead of classifying them as an attack. By training the model with only injecting backdoor triggers into benign traffic, we simulate a real-world scenario where an attacker attempts to manipulate the model's behavior without directly accessing attack traffic data.

A model is backdoored by training it with samples of clean features from both benign and attack packets, along with a random sample (backdoor percentage) of poisoned features of benign packets from a particular device (source IP-based). In our experiment, we assumed a device with a certain IP address carries out the poisoned trigger packet injection at a time. If the original packets from such a device are too low in number, then the number of poisoned packets also will remain low for that device. Our typical backdoor percentages are in the range of 0.5–10%. During testing time, we use a test sample of attack packets with a trigger.

Our backdoor approach is based on poisoning the features by injecting packets between real traffic. The backdoor trigger packets are created by arbitrarily fixing the destination IP and MAC address while keeping the realistic source IP of a device in the attack PCAP log. We retain the TCP port and frame length of the realistic packet from that device. Since the features are generated for a channel related to IP, our approach can cause changes to the features corresponding to succeeding normal packets after a trigger injection.

In all our experiments, we fix the training and testing ratio to 80–20. Due to the large volume of data used for training in the case of a multi-class classification model, we use a dataset of three attacks and corresponding benign datasets, and trim the dataset to a maximum of one million datapoints per class of attacks. For Fuzzing and Wiretapping datasets, we take 600,000 benign and 1,000,000 attack packets. Since the Mirai dataset is comparatively smaller, we take the complete benign and attack dataset for training.

4.4 Metrics

Attack Success Rate (ASR). ASR is defined as the ratio of samples with triggers misclassified by the backdoored model to the total number of samples with triggers used in the attack.

Silhouette score is a metric used to calculate how good the clustering technique is. We use the Silhouette score to evaluate the stealthiness of our backdoor mechanism. It ranges from −1 to 1,

where 1 indicates well-separated and clearly distinguished clusters, while 0 suggests that clusters are not well-separated, making it difficult to distinguish between data points from different clusters. In other words, a Silhouette score close to 1 indicates the optimal number of clusters for that dataset.

5 Results

5.1 Backdoor Performance

We use label-flipping poisoning attacks as a baseline. In the label-flipping attack, the adversary manipulates the malicious samples in the training data and changes the labels to benign. The baseline attack shows how much data needs to be manipulated during the training process to degrade the performance of the IDS model. When sufficient data is poisoned, the model misclassifies attack packets as benign. In contrast, we use a clean-label poisoning attack where the labels remain unchanged during the training process. Instead, we employ a backdoor trigger to activate the attack.

Table 1 shows our trigger-based backdoor approach's performance compared to the *label-flipping attack*, in the case of backdoor percentages of 2% and 50%. In the case of the *Label-flipping attack*, BD % indicates the percentage of labels flipped in the training dataset. As shown, the label-flipping attack requires a much larger portion of the dataset to be manipulated in order to succeed. In contrast, our approach requires a significantly smaller amount of trigger-poisoned data for the attack to be successful. For instance, to misclassify a CPS-ICMP flooding attack as benign, the baseline approach requires 35% of the dataset to be poisoned. In comparison, our approach achieves a 0.97 ASR by poisoning only 2% of the training dataset. Similarly, for the Modbus flooding attack, our PCAP-Backdoor technique can achieve an ASR of 0.98 with a backdoor percentage of 2%, while the baseline approach can achieve only a low ASR of 0.32 even when 50% of the training data labels are flipped. This demonstrates that the victim model can be effectively manipulated with significantly fewer poisoned samples in our method.

We further evaluate the performance of a multi-class IDS model in Table 1, which is designed to detect various types of network attacks. Here, the model's goal is not to identify malicious traffic, but to distinguish between different attack categories. Even in this case, our approach requires less data to achieve successful backdoor attacks. Specifically, when the trigger is present, the model misclassifies malicious traffic as benign. We also observe that our technique does not perform well on the Mirai dataset—presumably because it is a smaller dataset collected from too many devices, with each device contributing a relatively smaller number of packets compared to other attack datasets. As we discuss, we show that we can improve the backdoor performance on the Mirai attack by reducing the time delay between the trigger packets.

Takeaway. Our PCAP-Backdoor technique outperforms the baseline technique in terms of achieving high ASR while maintaining a very low trigger percentage in all datasets, while testing with both binary and multi-classifiers.

5.2 Performance on Different Attack Types

Next, we compare the performance of our backdoor approach on different attack types within the anomaly detection dataset for various devices. For this evaluation, we backdoor the benign traffic of one of the devices and analyze whether the IDS anomaly detection misclassifies anomalous traffic. We introduce trigger packets into the malicious data and measure the model's performance.

Figure 6 presents a box plot of the performance across different attack types in the dataset. Unless specified, we poison data from only a single device during the model training process.

When we present data with a trigger, the model misclassifies the input. This demonstrates that triggers alter the model's behavior, leading to incorrect classifications. Furthermore, our results

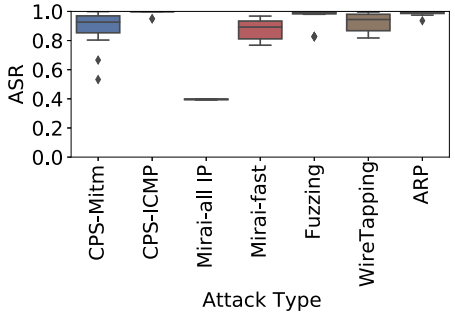
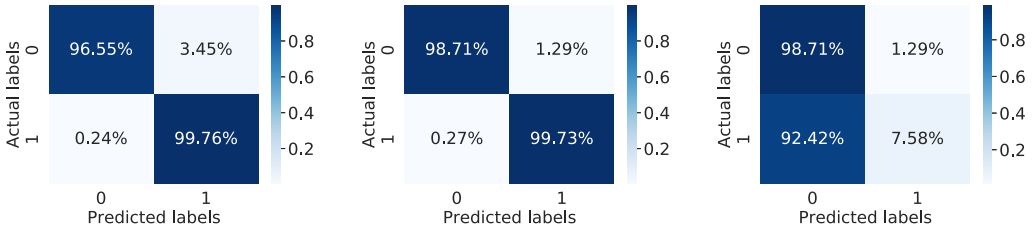


Fig. 6. Model performance on various attack types.



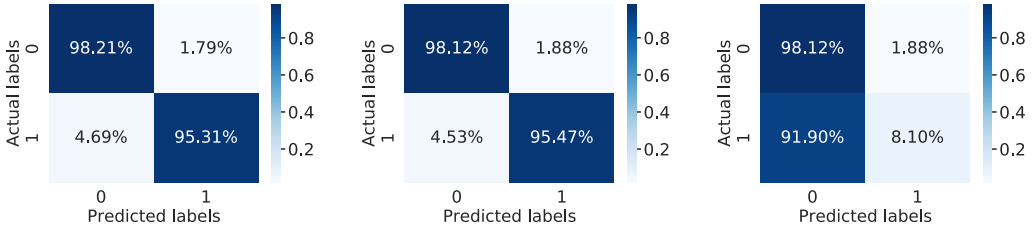
(a) Original data on honest model (b) Original data on backdoor model (c) Data with trigger on backdoor model

Fig. 7. Confusion matrix on the MITM attack in the CPS dataset.

demonstrate that even when we poison input samples from a single malicious device during training, we can effectively influence the model to misclassify traffic. Notably, in this scenario, the attacker does not have access to traffic originating from other devices. Despite this limitation, by injecting triggers into the malicious traffic from other devices, the model tends to classify this traffic as benign, as evidenced by a high ASR score. Additionally, the low variance in performance indicates that the backdoor attack consistently works across traffic from different devices.

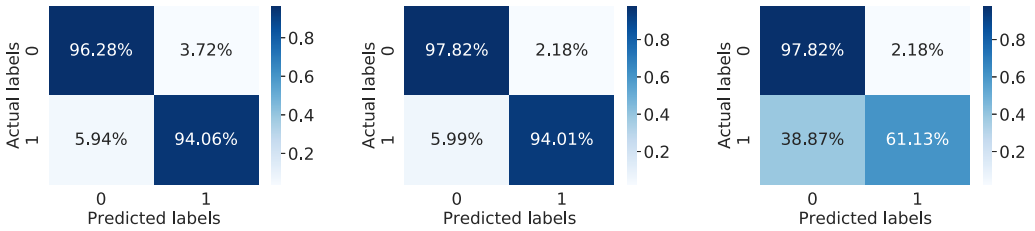
While the Mirai dataset does not perform well in backdoor attacks if only one device is infected, likely due to limited packets contributed by a single device, we observe that including traffic from all IP addresses improves the ASR. We also observe that the backdoor performance can be further enhanced by reducing the time delay between the trigger packets by up to one-tenth, as indicated by Mirai-fast in Figure 6, which demonstrates the influence of the trigger on the packet flow features.

We also analyze the behavior of the backdoor model on original data and poisoned data containing a trigger, depicted in Figures 7 and 8. We observe that when the data has no trigger (i.e., original data), the model can still identify benign and attack traffic with high accuracy (see Figures 7(b) and 8(b)). In contrast, with poisoned data with a trigger, the model classifies benign samples with triggers as benign, as shown in Figures 7(c) and 8(c). However, when we introduce the trigger on attack traffic, the model misclassifies it, even though the model was trained only on benign traffic with triggers and from a single device. This demonstrates that the backdoor attack successfully influences the model's behavior, causing it to misclassify attack traffic as benign. Figure 9 shows a similar set of confusion matrices of the Mirai attack. As explained previously, the backdoor attack does not perform well on the Mirai attack, but the performance can be improved by including trigger packets from more devices.



(a) Original data on honest model (b) Original data on backdoor model (c) Data with trigger on backdoor model

Fig. 8. Confusion matrix on the Fuzzing attack in the IoT dataset.



(a) Original data on honest model (b) Original data on backdoor model (c) Data with trigger on backdoor model

Fig. 9. Confusion matrix on the Mirai attack in the IoT dataset.

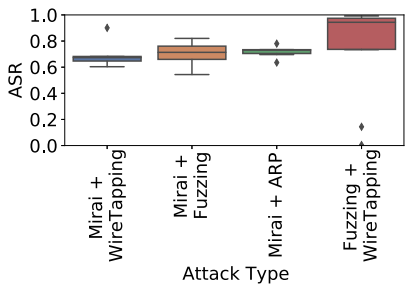


Fig. 10. Backdoor performance of different network attack data combinations.

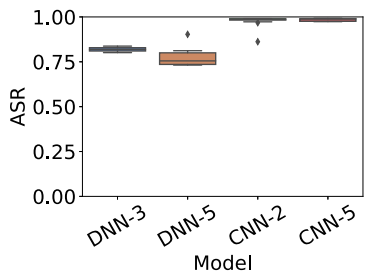


Fig. 11. Performance of various models on the Fuzzing attack dataset.

Takeaway. Backdoor attack is successful even when the attacker modifies benign traffic from a single device. By introducing the trigger during training on benign traffic, the model misclassifies malicious traffic as benign when the trigger is present during inference.

5.3 Effect of Multiple Attack Combination

In this experiment, we combine traffic from two different network attack classes. Specifically, we use various combinations of network attacks and label them as malicious if they consist of a network attack; otherwise, we classify them as benign. We then train our model using this combined dataset. Figure 10 shows the performance on different combinations of attack datasets across all IP addresses, varying the backdoor percentage from 0.5% to 10%. As depicted in the figure, the high ASR indicates that the model misclassifies malicious attacks as benign. In particular, we observe that the median ASR score when we combine Mirai+WireTapping is 0.63.

Takeaway. Backdoor attacks perform well even if the dataset contains traffic from multiple attacks.

5.4 Impact on Different IDS Models

We now evaluate the efficacy of introducing a backdoor into various neural network intrusion detection models. Our approach involves designing a range of models of varying sizes, denoted by DNN-5, indicating a model with five layers. Additionally, we explore CNN-based neural networks as part of our model architecture. All these models, when trained with original data, produce high accuracy.

In order to evaluate the backdoor technique on different model architectures, we train all these models with a 10% percentage of poisoned data, and the trigger packet count is 3. The impact of training with our backdoor technique on the Fuzzing attack is illustrated in Figure 11. We observe that the different IDS models are vulnerable to backdoor attacks, as indicated by high ASR scores. However, when presented with original data with no triggers containing both benign and malicious traffic, all models achieve a high accuracy of at least 99%. This indicates that the model predicts correctly when no trigger is present. We also observe that CNN-based IDS models are susceptible to backdoor attacks, indicated by the high ASR.

Takeaway. The backdoor attack is effective on different neural network-based IDS models. This emphasizes the need for robust defenses against such manipulations.

5.5 Multi-Class Classification of Multiple Attacks

We also analyze the backdoor performance on a multi-class classification model, where the model identifies one of the four classes: Mirai, Fuzzing, Wiretapping, and Benign. For this experiment, as before, we set the trigger packet count to 5 and set the backdoor percentage rate to 1%. In other words, the attacker contributes only 1% for training the model. Figure 14 depicts the confusion matrix for the model's behavior when given original data and when given our backdoor data with triggers. We note that the honest model performs well with an F1-score of 0.94 or more, as shown in Figure 14(a). Moreover, as shown in Figure 14(b), the model behaves normally and identifies malicious traffic when no trigger is present. However, when we introduce our trigger-based malicious traffic, the model misclassifies it as benign, as seen in Figure 14(c). This indicates that the attacker can successfully create backdoors even for a multi-class classification model.

Takeaway. Backdoor attack works on a multi-class classification model even when we have only a single infected device and 1% of backdoor data.

5.6 Effect of Using Different Features

In this experiment, we explore different feature extractors by modifying the input features used for training. While our initial feature extractor is based on Kitsune [36], we consider other distinct feature sets. In particular, the first set contains features related to jitter. The next feature extractor is based on packet size-related features. We also create a feature extractor that contains only socket-related features as our third set. Finally, the fourth set consists of all 115 features used in Kitsune. Next, we focus on using these different feature extractors on the fuzzing dataset with a trigger packet count of 3. The results, shown in Figure 12, reveal that the backdoor attack achieves high ASR scores across all feature sets. In the case of a CPS-Mitm attack, the average ASR is 0.86 for the jitter feature alone, 0.91 for the packet size feature alone, and 0.92 for all features. Interestingly, a similar pattern was observed with the Fuzzing dataset, indicating that the backdoor attack performs best when using the feature set related to packet size. This indicates that our backdoor attack can effectively influence the model's behavior regardless of the specific feature extraction method.

Takeaway. Despite variations in backdoor performance across different feature extractors, the attack remains effective overall. Interestingly, we found that packet-size-related features are more susceptible to backdoor attacks compared to other feature sets.

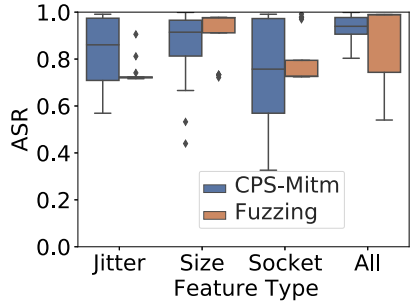


Fig. 12. Performance on various feature sets.

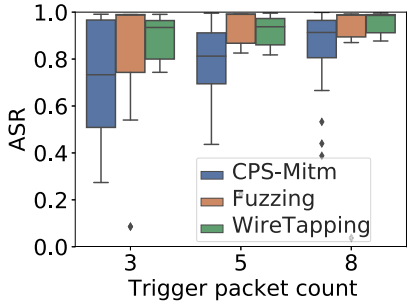
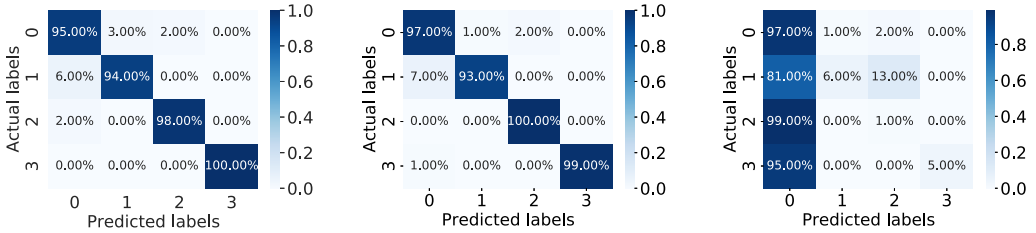


Fig. 13. Performance on varying packet count.



(a) Original data on honest model (b) Original data on backdoor model (c) Data with trigger on backdoor model

Fig. 14. Confusion matrix for the multi-class classification model.

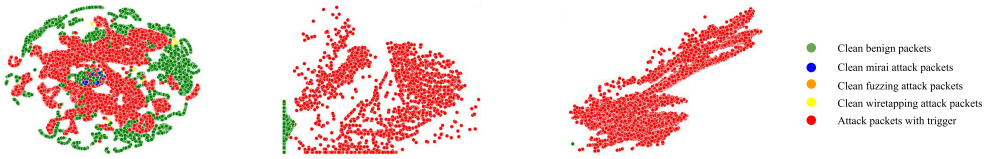
5.7 Effect of Backdoor Trigger Packet Count

We now assess the effect of trigger packet count on backdoor performance. Note that trigger packet count controls the number of consecutive packets introduced during backdoor generation. We report our analysis for different backdoor percentages varying from 0.5% to 10%. Figure 13 illustrates the impact on performance for varying trigger packet counts and different datasets. As depicted in the figure, we observe that as the trigger packet count increases, the ASR score increases. This indicates that the attacker can successfully change the model's behavior with a higher trigger packet count. Intuitively, introducing more consecutive packets exerts a larger influence on flow-based statistics extracted from the feature extractor, enabling the attacker to effectively inject the trigger and execute the backdoor attack.

Takeaway. Increasing the backdoor trigger packet count during backdoor generation can improve the success of backdoor attacks.

6 Backdoor Attack Defense

There are several proposed works on defending against backdoor attacks, but they have mostly been analyzed in the vision domain [7, 17, 52, 54]. Cleansing techniques based on Self-Attenuation Distillation have their limitations. They are most effective when applied to deeper neural networks and have primarily been studied in the context of specific network types, like CNNs [17]. In contrast, PCAP-Backdoor is effective for both shallow and deep networks. Moreover, NeuralCleanse works by reverse-engineering static triggers, with the intuition that an attacker tries to make a minimal perturbation to misclassify an input into a target class [52]. Whereas, PCAP-Backdoor considers a bidirectional packet flow scenario with temporal features calculated based on incremental statistics. In this context, we examine the following well-known backdoor defense techniques in the packet domain.



(a) Combined (b) Activation Cluster 1 (c) Activation Cluster 2

Fig. 15. Activations of the last hidden layer of the backdoored model on packets without a trigger and attack packets with a trigger classified as benign are shown in (a). Two activation clusters of benign predictions by the backdoored model are shown in (b) and (c), indicating no distinct clusters formed by actual benign and misclassified benign predictions by the backdoored model. Misclassified benign predictions due to the presence of triggers are spread across both clusters.

6.1 Activation Clustering Analysis

We now analyze whether we can detect the presence of backdoor triggers using the activation clustering algorithm [7, 39]. The basic idea is to cluster the last hidden layer's activations from benign and poisoned attack samples (with triggers) that were classified as benign by the backdoored model. The hypothesis is that the presence of triggers will cause the poisoned samples to activate distinct neurons, leading to the formation of two separate clusters in the activation space—one cluster with the activations from the benign samples and another cluster with the activations from the poisoned samples. The poisoned samples, influenced by the backdoor trigger, will cause the model to activate a different set of neurons compared to the benign samples. As a result, despite both being classified as benign, the activations of poisoned samples will form a separate group in the activation space, distinct from the group formed by the normal benign samples. The formation of two distinct clusters indicates that the model processes poisoned and benign samples differently, indicating the presence of a backdoor.

We follow the activation clustering algorithm described in [7]. First, for each sample, we extract the activation from the last hidden layer of the IDS classifier model and flatten it into a single 1-dimensional vector. Next, for each activation vector, we apply the PCA algorithm to reduce the dimensionality of the data. Then, we perform k-means clustering with a cluster size of 2 on the reshaped data, grouping the activations based on their similarity. Finally, we analyze the results for any anomalies by checking whether the clusters reveal distinct groupings, which may indicate the presence of poisoned or backdoored samples. For more details, please refer [7]. For our evaluation, we analyze a multi-class model with four attack classes: Mirai, Fuzzing, Wiretapping, and Benign. Note that while we analyzed the multi-class model, we also analyzed the single-class model, which yielded similar results. Figure 15(a) shows the activation of the last hidden layer for the dataset samples that were predicted as benign by the backdoored model [51]. If the defense is effective, we expect to observe two distinct clusters, where benign samples form one cluster and attack samples with triggers form another. However, as seen in Figure 15(b) and (c), this is not the case. The attack samples with triggers are distributed across different clusters, indicating the absence of clear clusters.

To assess clustering quality, we compute Silhouette scores for various cluster sizes. Ideally, the highest score should correspond to a cluster size of two, indicating well-separated clusters—benign and attack with trigger. In contrast, our observations reveal that the Silhouette score peaks at a cluster size of three, followed by four, suggesting a lack of well-defined clusters. We also use the Silhouette score to measure the stealthiness of our backdoored model quantitatively. Specifically, we aim to compare and identify the Silhouette score for different cluster sizes, focusing on understanding the effectiveness of a cluster size of 2 compared to other cluster configurations. To

Table 2. Silhouette Scores for Each Cluster Size

Cluster size	1	**2**	**3**	4	5	6	7
Silhouette scores	NA	0.62	**0.73**[a]	0.72	0.66	0.68	0.67

[a]The silhouette score peaks at three clusters, indicating that normal benign packets and trigger-based attack packets cannot be accurately separated into two clusters.

Table 3. Impact of Adversarial Training on Our Attack Model on CPS MITM Attack

Metric	Before adversarial training	After adversarial training with different % of adversarial samples		
		5%	10%	20%
Clean accuracy	99.8	99.5	98.7	96.1
ASR	0.92	0.88	0.85	0.79

achieve this, we compute the Silhouette score on the hidden layer output of our backdoored model, corresponding to predictions of the benign class. Table 2 shows the silhouette scores corresponding to each cluster. We observe that the Silhouette score for cluster number 2 is low compared to other clusters. This suggests that distinguishing between normal-benign and attack with trigger samples based on the hidden layer output is challenging, emphasizing the effectiveness of the backdoor in obfuscating model behavior [7].

Takeaway. Activation clustering-based algorithm is unable to detect the presence of backdoor triggers in the model. Specifically, the absence of two distinct clusters suggests that the activations were not distinct enough to be successfully detected using this method.

6.2 Adversarial Training

Adversarial training is a model hardening-based defense technique against adversarial attacks [39]. In this technique, the model is retrained with adversarial samples labeled correctly instead of the target class. Once the model is trained, it is expected to predict poisoned data correctly instead of misclassifying it as the target class. We apply an adversarial training-based technique on our backdoored model by incorporating adversarial samples of the non-target class labeled as its actual class in the training set. The results are shown in Table 3. We carry out our experiment on different percentages of adversarial samples used in the non-target class training data. As shown, ASR reduces when the number of adversarial samples increases in training data. For instance, in the case of an MITM attack, with 20% adversarial samples, the adversarially trained model achieves an ASR reduction from 0.92 to 0.79. However, the clean accuracy (data without trigger on the backdoored model) also drops to 96.1% from 99.8%. While adversarial training helps in hardening the model by incorporating a large amount of adversarial samples, it is challenging to generate such adversarial samples and also requires knowing the entire set of strategies that the adversary may adopt.

Takeaway. Adversarial training reduces ASR but decreases clean accuracy. Additionally, generating adversarial samples requires knowledge of the backdoor training process.

6.3 Neural Pruning

The neural pruning-based backdoor defense technique works by eliminating neurons in the backdoor model that are minimally activated when presented with a clean input. We follow the neural pruning-based defense technique similar to [3, 27, 32]. Neurons in the shallow layers are typically able to capture macro features, while the deep layers capture the micro features of the data [27]. Neural pruning can be performed locally at a certain layer by randomly choosing some set of

Table 4. Impact of Pruning on Our Trigger Generation Technique on the MITM and Fuzzing Attacks

Attack	Performance metric	Our attack	Local pruning	Global pruning
CPS MITM	Attack ASR	0.92	0.89	0.90
	Clean accuracy (%)	99.8	95.3	96.1
IoT fuzzing	Attack ASR	0.92	0.87	0.87
	Clean accuracy (%)	95.3	91.6	91.9

neurons, or globally by randomly selecting some set of neurons across layers of the neural network. We consider both local pruning and global pruning approaches, identifying the most impacted set of neurons to be pruned iteratively, and the results are plotted in Table 4. As shown, the table presents both clean accuracy and backdoor performance in ASR for both local pruning and global pruning. While both pruning techniques reduce the attack ASR, they also result in a decrease in model accuracy. For instance, in the case of CPS MITM, global pruning reduces the attack ASR from 0.92 to 0.90, and the clean accuracy (data without trigger on backdoored model) also drops by 3.7%. A similar reduction in accuracy is observed with an IoT fuzzing attack as well.

Takeaway. Pruning-based defense technique is challenging in a shallow neural network. In the case of deep network models trained on PCAP data, pruning results in reducing the attack success at the expense of clean accuracy.

7 Discussion

In our work, we ensure that the attacker provides a traffic dataset from only one device. However, we observe that introducing traffic from other devices can further improve the accuracy of the backdoor attack. Interestingly, we also found that the backdoor percentage data has a limited influence on performance. That is, increasing the backdoor percentage does not lead to a linear improvement in backdoor performance. This observation was also made when we introduced malicious samples as benign for training in our baseline comparison. Only after a certain backdoor data threshold do we see the impact of the backdoor percentage taking effect. While without triggers, this backdoor threshold is high, our trigger-based approach enables the threshold to be decreased. Moreover, we noticed that changing both the destination and source ports does not significantly influence the overall performance. This observation suggests that altering unique packets unrelated to the source does not significantly impact flow-based statistics. However, a detailed analysis of other possible changes that could introduce a backdoor is left for future work.

While our current approach demonstrates that a simple packet injection at predefined intervals works well, possible future extensions could explore using intelligent generative adversarial networks to introduce these packets [13]. Leveraging GANs could potentially create more sophisticated backdoor attacks, posing additional challenges in detecting these backdoors. Finally, detecting these triggers using activation-based clustering techniques is challenging. Future work could include developing and exploring mitigation techniques to counter these types of backdoor attacks effectively.

8 Related Work

There have been numerous studies on adversarial attacks in the context of images, audio, and text [9, 20, 26, 33, 40, 42, 53]. One of the earliest works on adversarial example generation based on the fast gradient sign technique is proposed by Goodfellow et al. [18]. Cleverhans [43] is a tool that combines various adversarial example generation techniques that can be used to develop more robust ML models. Such adversarial attack techniques are majorly studied in the computer

vision domain for image classification [19], and the adversarial examples generated can be used for adversarial training. Still, they cannot be directly utilized for performing targeted attacks. Targeted adversarial attacks involve adding a backdoor to the model by poisoning input data with triggers and labeling them as the target class during the training process [20]. The model, thus trained, tends to misclassify input data into the specified target label when a trigger is present.

Prior work has also focused on generating triggers to allow adversaries to manipulate the model behavior [33, 37]. For example, Trojan Attack [33] inverses the model to generate a trojan trigger and retrains the model with reverse-engineered training data. This allows the attack to be carried out without the original training data. Similarly, Nguyen and Tran [37] propose an autoencoder-based backdoor generation technique to generate input-aware backdoors on images. This enables the triggers to be different for each input. There has also been work focusing on specific threat models. For example, a clean label backdoor attack [40] poisons training samples without altering their label, and during inference, the attacker applies the same trigger to cause misclassification. Recent work also demonstrates a dynamic black-box backdoor attack on IoT sensory data without having access to the model training process [3]. We adopt this threat model in our work.

While some sophisticated techniques exist to make triggers visually imperceptible [10, 11] or require access to the original dataset used for training the model to introduce the backdoor [33], our work takes a different approach. We focus on injecting backdoors into network traffic models without directly manipulating the features used by the model. Despite the intermediate processing involved in network traffic analysis, our approach demonstrates its effectiveness in introducing backdoors in various scenarios. Trigger-based attacks on network traffic flow are one of the least explored attacks in this class, and we cannot directly use the trigger techniques from the computer vision domain in a network packet-based dataset, mainly because the features used for training both models are of different characteristics. In the case of computer vision models, we pass image pixels to the deep neural networks, such as CNN, but in the case of the network traffic domain, we generally use traffic flow-based statistical features in the time or frequency domain or packet payload-based features [56]. Our technique works by injecting backdoor packets into raw network packets while considering the constraints imposed by networking protocols on the amount of perturbation that can be made on network packets to preserve the communication standards or protocols used in network communication.

Table 5 summarizes the key differences compared to prior work. Prior work has explored poisoning the network flow features, which requires access to the feature extractor [21, 22, 41]. For example, TrojanFlow [41] focuses on poisoning the packet size feature. Compared to existing backdoor techniques in the IDS domain, we find ours is a new class of attack that operates directly on raw network packets, rather than on extracted features. Furthermore, we implement a clean label poisoning attack, which assumes that the attacker has control over the data collection process but does not alter the ground truth labels, making the attack more realistic. Additionally, our method requires only a small number of poisoned packets, making it more efficient than prior approaches. Importantly, while prior work often assumes the need to poison data from multiple clients, our approach achieves its effectiveness by poisoning only a single client. The data poisoning process is designed to preserve the underlying network protocol communication to enhance its stealthiness. We evaluate our approach on IDS datasets from both IoT and SCADA, demonstrating its broader applicability. While various identification [7] and mitigation techniques [8, 16, 17, 23, 26, 29, 46, 47, 49, 52, 54, 58] have been proposed to defend against neural backdoors, these approaches are complementary to our work and are left for future exploration in the context of network intrusion detection.

Table 5. Comparison with Prior Work

Technique	Domain	Input raw/features	% Data poisoned for successful attack (client access)	Attack technique	Protocol-aware
TrojanFlow [41]	Traffic classification (IoT)	Feature	20% (NA)	Generator-based packet size modification	NA
Nguyen et al. [38]	IDS (IoT)	NA	30–40% (multiple)	Label flipping attack of non-target class	NA
VulnerGAN [31]	IDS (IoT)	Features	NA (NA)	GAN-based trigger generator using a shadow model	No
Jinhyeok et al. [24]	IDS (IoT)	Features	5% (multiple)	Poisoning high-importance features and label flipping of non-target class	No
DPOT [57]	Vision	Features	50% (multiple)	Data poisoning attack based on trigger placement algorithm	No
PCAP-Backdoor (Ours)[a]	IDS (IoT + **SCADA**)	**Raw packets**	**1%** (single client)	Trigger packet injection on **target** class data w/o violating the protocol	**Yes**

[a] PCAP-Backdoor is effective on IoT network and SCADA datasets, by injecting backdoor on raw packets of target class from a single client.

9 Conclusion

In this article, we introduce PCAP-Backdoor, a novel system for injecting backdoors in DL-based network intrusion detection models. Our threat model assumes that the attacker cannot access the feature extractor, making the backdoor injection challenging. Our technique injects a targeted backdoor into raw packets, requiring the careful crafting of backdoor trigger packets that adhere to the underlying network protocol. Despite these challenges, our experiments demonstrate that an attacker can effectively backdoor the model by contributing only poisoned benign traffic during model training. Our extensive evaluations of multiple network attack datasets, models, and feature extractors show that our backdoor injection technique performs well under various conditions. In particular, the attacker can successfully carry out the attack even with a poisoned dataset containing 1% or fewer poisoned samples. Furthermore, we demonstrate that the attack is difficult to detect when analyzed using activation-based clustering techniques. We also evaluate the attack's success in the context of adversarial training and pruning-based defense techniques, and demonstrate that the attack is not easily defended without compromising clean accuracy or understanding the knowledge of the backdoor technique.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback and constructive comments, which have helped improve the quality of this work.

References

[1] Pallavi Asrodia and Hemlata Patel. 2012. Analysis of various packet sniffing tools for network monitoring and analysis. *International Journal of Electrical, Electronics and Computer Engineering* 1, 1 (2012), 55–58.

[2] Ajesh Koyatan Chathoth, Abhyuday Jagannatha, and Stephen Lee. 2021. Federated intrusion detection for IoT with heterogeneous cohort privacy. arXiv:2101.09878. Retrieved from https://arxiv.org/abs/2101.09878

[3] Ajesh Koyatan Chathoth and Stephen Lee. 2024. Dynamic black-box backdoor attacks on IoT sensory data. In *Proceedings of the 2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*. IEEE, 182–191.

[4] Ajesh Koyatan Chathoth, Clark P. Necciai, Abhyuday Jagannatha, and Stephen Lee. 2022. Differentially private federated continual learning with heterogeneous cohort privacy. In *Proceedings of the 2022 IEEE International Conference on Big Data (Big Data)*. IEEE, 5682–5691.

[5] Ajesh Koyatan Chathoth, Shuhao Yu, and Stephen Lee. 2025. Dynamic user-controllable privacy-preserving few-shot sensing framework. arXiv:2508.03989. Retrieved from https://arxiv.org/abs/2508.03989

[6] Ajesh Koyatan Chathoth, Shuhao Yu, and Stephen Lee. 2025. PrivCLIP: Dynamic user-controllable privacy-preserving few-shot sensing framework. In *Proceedings of the 2025 IEEE International Conference on Big Data (BigData)*. IEEE, 1793–1798.

[7] Bryant Chen, Wilka Carvalho, Nathalie Baracaldo, Heiko Ludwig, Benjamin Edwards, Taesung Lee, Ian Molloy, and Biplav Srivastava. 2018. Detecting backdoor attacks on deep neural networks by activation clustering. arXiv:1811.03728. Retrieved from https://arxiv.org/abs/1811.03728

[8] Weixin Chen, Baoyuan Wu, and Haoqian Wang. 2022. Effective backdoor defense by exploiting sensitivity of poisoned samples. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (NIPS '22)*, 9727–9737.

[9] Jiazhu Dai, Chuanshuai Chen, and Yufeng Li. 2019. A backdoor attack against LSTM-based text classification systems. *IEEE Access* 7 (2019), 138872–138878. DOI: https://doi.org/10.1109/ACCESS.2019.2941376

[10] Khoa Doan, Yingjie Lao, and Ping Li. 2021. Backdoor attack with imperceptible input and latent modification. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*, 18944–18957.

[11] Khoa Doan, Yingjie Lao, Weijie Zhao, and Ping Li. 2021. Lira: Learnable, imperceptible and robust backdoor attacks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 11966–11976.

[12] Yuansheng Dong, Rong Wang, and Juan He. 2019. Real-time network intrusion detection system based on deep learning. In *Proceedings of the 2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, 1–4. DOI: https://doi.org/10.1109/ICSESS47205.2019.9040718

[13] Baik Dowoo, Yujin Jung, and Changhee Choi. 2019. PcapGAN: Packet capture file generator by style-based generative adversarial networks. In *Proceedings of the 2019 18th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 1149–1154.

[14] Ivo Frazão, Pedro Henriques Abreu, Tiago Cruz, Hélder Araújo, and Paulo Simões. 2019. Denial of service attacks: Detecting the frailties of machine learning algorithms in the classification process. In *Proceedings of the 13th International Conference on Critical Information Infrastructures Security (CRITIS '18)*. Springer, 230–235.

[15] Leilei Gan, Jiwei Li, Tianwei Zhang, Xiaoya Li, Yuxian Meng, Fei Wu, Yi Yang, Shangwei Guo, and Chun Fan. 2021. Triggerless backdoor attack for NLP tasks with clean labels. arXiv:2111.07970. Retrieved from https://arxiv.org/abs/2111.07970

[16] Kuofeng Gao, Yang Bai, Jindong Gu, Yong Yang, and Shu-Tao Xia. 2023. Backdoor defense via adaptively splitting poisoned dataset. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 4005–4014.

[17] Xueluan Gong, Yanjiao Chen, Wang Yang, Qian Wang, Yuzhe Gu, Huayang Huang, and Chao Shen. 2023. Redeem myself: Purifying backdoors in deep learning models using self attention distillation. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 755–772.

[18] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. arXiv:1412.6572. Retrieved from https://arxiv.org/abs/1412.6572

[19] Dou Goodman, Hao Xin, Wang Yang, Wu Yuesheng, Xiong Junfeng, and Zhang Huan. 2020. Advbox: a toolbox to generate adversarial examples that fool neural networks. arXiv:2001.05574. Retrieved from https://arxiv.org/abs/2001.05574

[20] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access* 7 (2019), 47230–47244.

[21] John T. Holodnak, Olivia Brown, Jason Matterer, and Andrew Lemke. 2022. Backdoor poisoning of encrypted traffic classifiers. In *Proceedings of the 2022 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 577–585.

[22] Yongjin Hu, Jin Tian, and Jun Ma. 2021. A novel way to generate adversarial network traffic samples against network traffic classification. *Wireless Communications and Mobile Computing* 2021 (2021), 7367107.

[23] Xijie Huang, Moustafa Alzantot, and Mani Srivastava. 2019. NeuronInspect: Detecting backdoors in neural networks via output explanations. arXiv:1911.07399. Retrieved from https://arxiv.org/abs/1911.07399

[24] Jinhyeok Jang, Yoonsoo An, Dowan Kim, and Daeseon Choi. 2023. Feature importance-based backdoor attack in NSL-KDD. *Electronics* 12, 24 (2023), 4953.

[25] Ahmad Javaid, Quamar Niyaz, Weiqing Sun, and Mansoor Alam. 2016. A deep learning approach for network intrusion detection system. In *Proceedings of the 9th EAI International Conference on Bio-Inspired Information and Communications Technologies (Formerly BIONETICS)*, 21–26.

[26] Wenbo Jiang, Hongwei Li, Guowen Xu, and Tianwei Zhang. 2023. Color backdoor: A robust poisoning attack in color space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8133–8142.

[27] Artur Jordao and Hélio Pedrini. 2021. On the effect of pruning on adversarial robustness. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1–11.

[28] Nagendra Kumar, Yogesh Ramdoss, and Yoram Orzach. 2018. *Network Analysis Using Wireshark 2 Cookbook: Practical Recipes to Analyze and Secure Your Network Using Wireshark 2*. Packt Publishing Ltd.

[29] Yige Li, Xixiang Lyu, Nodens Koren, Lingjuan Lyu, Bo Li, and Xingjun Ma. 2021. Anti-backdoor learning: Training clean models on poisoned data. In *Proceedings of the 35th Conference on Neural Information Processing Systems (NIPS '21)*, 14900–14912.

[30] Yanjie Li, Zhen Xiang, Nathaniel D. Bastian, Dawn Song, and Bo Li. 2024. IDS-Agent: An LLM agent for explainable intrusion detection in IoT networks. In *Proceedings of the Workshop on Open-World Agents: Synnergizing Reasoning and Decision-Making in Open-World Environments (OWA '24)*.

[31] Guangrui Liu, Weizhe Zhang, Xinjie Li, Kaisheng Fan, and Shui Yu. 2022. VulnerGAN: A backdoor attack through vulnerability amplification against machine learning-based network intrusion detection systems. *Science China Information Sciences* 65, 7 (2022), 170303.

[32] Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2018. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*. Springer, 273–294.

[33] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. 2018. Trojaning attack on neural networks. In *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS '18)*. The Internet Society.

[34] Yair Meidan, Michael Bohadana, Yael Mathov, Yisroel Mirsky, Asaf Shabtai, Dominik Breitenbacher, and Yuval Elovici. 2018. N-BaIoT—Network-based detection of IoT botnet attacks using deep autoencoders. *IEEE Pervasive Computing* 17, 3 (2018), 12–22.

[35] Maksym Melnyk, Jacob Thomas, Max Wandera, Ajesh Koyatan Chathoth, and Michael Zuzak. 2025. Hardware anomaly detection in microcontrollers through watchdog-assisted property enforcement. In *Proceedings of the 2025 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE, 1–6.

[36] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An ensemble of autoencoders for online network intrusion detection. arXiv:1802.09089. Retrieved from https://arxiv.org/abs/1802.09089

[37] Tuan Anh Nguyen and Anh Tran. 2020. Input-aware dynamic backdoor attack. In *Proceedings of the 34th Conference on Neural Information Processing Systems (NIPS '20)*, 3454–3464.

[38] Thien Duc Nguyen, Phillip Rieger, Markus Miettinen, and Ahmad-Reza Sadeghi. 2020. Poisoning attacks on federated learning-based IoT intrusion detection system. In *Proceedings of the Workshop on Decentralized IoT Systems and Security (DISS)*, 1–7.

[39] Maria-Irina Nicolae, Mathieu Sinn, Minh Ngoc Tran, Beat Buesser, Ambrish Rawat, Martin Wistuba, Valentina Zantedeschi, Nathalie Baracaldo, Bryant Chen, Heiko Ludwig, et al. 2018. Adversarial robustness toolbox v1. 0.0. arXiv:1807.01069. Retrieved from https://arxiv.org/abs/1807.01069

[40] Rui Ning, Jiang Li, Chunsheng Xin, and Hongyi Wu. 2021. Invisible poison: A blackbox clean label backdoor attack to deep neural networks. In *Proceedings of the IEEE Conference on Computer Communications (IEEE INFOCOM '21)*, 1–10. DOI: https://doi.org/10.1109/INFOCOM42981.2021.9488902

[41] Rui Ning, Chunsheng Xin, and Hongyi Wu. 2022. TrojanFlow: A neural backdoor attack to deep learning-based network traffic classifiers. In *Proceedings of the IEEE Conference on Computer Communications (IEEE INFOCOM '22)*. IEEE, 1429–1438.

[42] Xudong Pan, Mi Zhang, Beina Sheng, Jiaming Zhu, and Min Yang. 2022. Hidden trigger backdoor attack on NLP models via linguistic style manipulation. In *Proceedings of the 31st USENIX Security Symposium (USENIX Security ' 22)*, 3611–3628.

[43] Nicolas Papernot, Fartash Faghri, Nicholas Carlini, Ian Goodfellow, Reuben Feinman, Alexey Kurakin, Cihang Xie, Yash Sharma, Tom Brown, Aurko Roy, et al. 2016. Technical report on the CleverHans v2. 1.0 adversarial examples library. arXiv:1610.00768. Retrieved from https://arxiv.org/abs/1610.00768

[44] PcapPlusPlus. 2024. PcapPlusPlus. Retrieved April 2024 from https://pcapplusplus.github.io/community

[45] Anfeng Peng, Ajesh Koyatan Chathoth, and Stephen Lee. 2025. Log anomaly detection with large language models via knowledge-enriched fusion. arXiv:2512.11997. Retrieved from https://arxiv.org/abs/2512.11997

[46] Ximing Qiao, Yukun Yang, and Hai Li. 2019. Defending neural backdoors via generative distribution modeling. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*.

[47] Xiaofei Sun, Xiaoya Li, Yuxian Meng, Xiang Ao, Lingjuan Lyu, Jiwei Li, and Tianwei Zhang. 2023. Defending against backdoor attacks in natural language generation. *Proceedings of the AAAI Conference on Artificial Intelligence* 37 (2023), 5257–5265.

[48] Simen Thys, Wiebe Van Ranst, and Toon Goedemé. 2019. Fooling automated surveillance cameras: Adversarial patches to attack person detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*.

[49] Brandon Tran, Jerry Li, and Aleksander Madry. 2018. Spectral signatures in backdoor attacks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS '18)*.

[50] tshark. 2024. Wireshark. Retrieved April, 2024 from https://www.wireshark.org/docs/man-pages/tshark.html

[51] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of Machine Learning Research* 9 (2008), 2579–2605.

[52] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. 2019. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *Proceedings of the 2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 707–723.

[53] Zhenting Wang, Juan Zhai, and Shiqing Ma. 2022. BppAttack: Stealthy and efficient trojan attacks against deep neural networks via image quantization and contrastive adversarial learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 15074–15084.

[54] Dongxian Wu and Yisen Wang. 2021. Adversarial neuron pruning purifies backdoored deep models. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*, 16913–16925.

[55] Kaidi Xu, Gaoyuan Zhang, Sijia Liu, Quanfu Fan, Mengshu Sun, Hongge Chen, Pin-Yu Chen, Yanzhi Wang, and Xue Lin. 2020. Adversarial t-shirt! evading person detectors in a physical world. In *Proceedings of the 16th European Conference on Computer Vision (ECCV '20)*. Springer, 665–681.

[56] Kun Yang, Samory Kpotufe, and Nick Feamster. 2020. Feature extraction for novelty detection in network traffic. arXiv:2006.16993. Retrieved from https://arxiv.org/abs/2006.16993

[57] Yujie Zhang, Neil Gong, and Michael K. Reiter. 2024. Concealing backdoor model updates in federated learning by trigger-optimized data poisoning. arXiv:2405.06206. Retrieved from https://arxiv.org/abs/2405.06206

[58] Mingli Zhu, Shaokui Wei, Hongyuan Zha, and Baoyuan Wu. 2024. Neural polarizer: A lightweight and effective backdoor defense via purifying poisoned features. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*.

[59] zscaler. 2023. Zscaler ThreatLabz 2023 Enterprise IoT and OT Threat Report. Retrieved November 2024 from https://info.zscaler.com/resources-industry-reports-threatlabz-2023-enterprise-ioT-ot-threat-report

Received 7 February 2025; revised 29 August 2025; accepted 13 March 2026